

Modelos de diseño base de datos (Sprint 1)

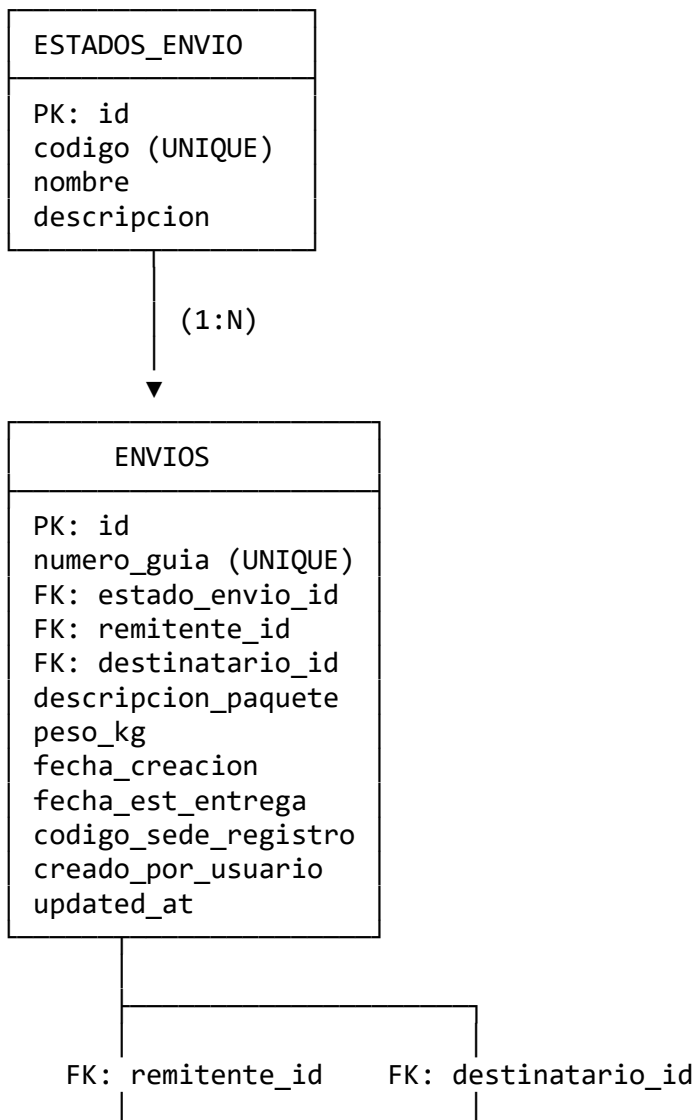
Estudiante: Juan Pablo Collante Dominguez

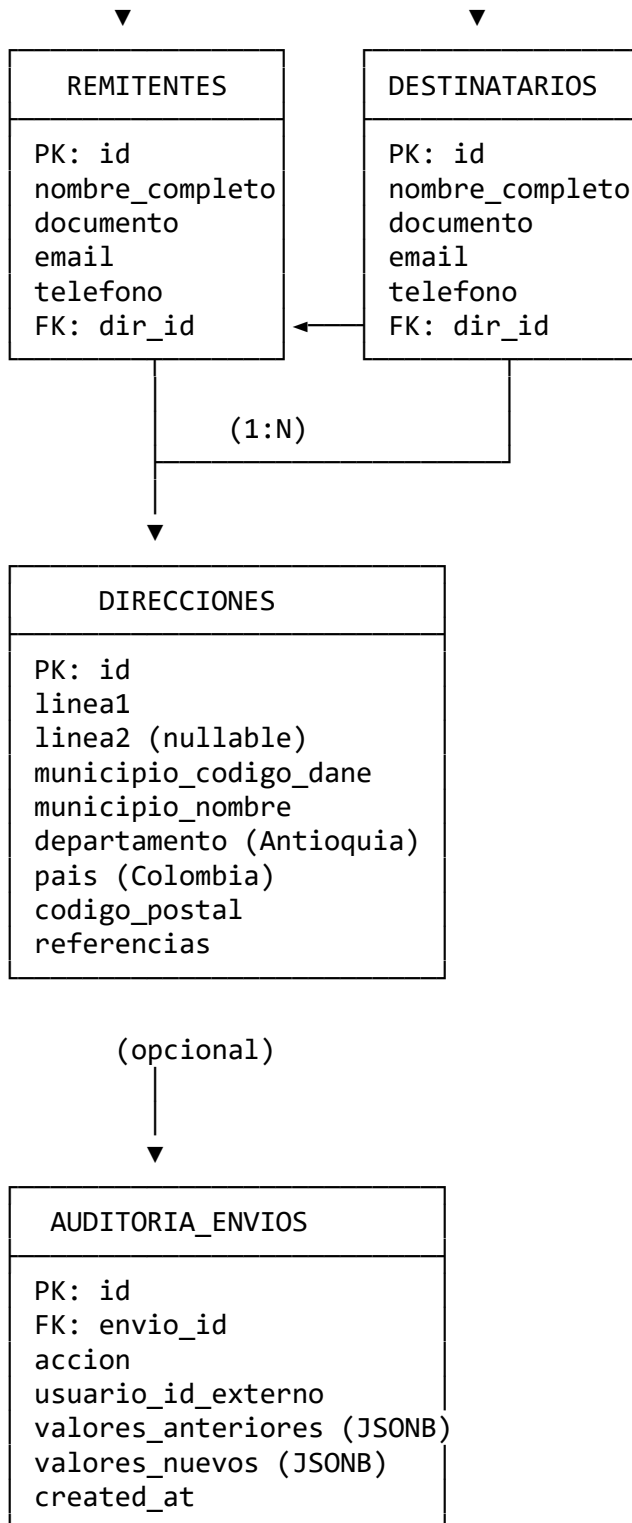
Cedula: 1019012461

Diagrama Entidad-Relación (DER)

Sistema de Gestión de Envíos - logistica-envios

1. Diagrama ER Conceptual





2. Matriz de Relaciones

Entidad A	Cardinalidad	Entidad B	Tipo	Descripción
ESTADOS_ENVIO	1:N	ENVIOS	Obligatoria	Un estado puede tener múltiples envíos
REMITENTES	1:N	ENVIOS	Obligatoria	Un remitente puede tener múltiples envíos (potencial)
DESTINATARIOS	1:N	ENVIOS	Obligatoria	Un destinatario puede tener múltiples envíos (potencial)
DIRECCIONES	1:N	REMITENTES	Obligatoria	Una dirección es del remitente
DIRECCIONES	1:N	DESTINATARIOS	Obligatoria	Una dirección es del destinatario
ENVIOS	1:N	AUDITORIA_ENVIOS	Opcional	Un envío puede tener múltiples registros de auditoría

3. Atributos por Entidad

ESTADOS_ENVIO

- **id** (PK, Auto-increment): Identificador único
- **codigo** (UNIQUE, Varchar(30)): CREADO, EN_TRANSITO, EN_SEDE, ENTREGADO, INCIDENCIA
- **nombre** (Varchar(100)): Nombre legible del estado
- **descripcion** (Text): Descripción detallada del estado

DIRECCIONES

- **id** (PK, Auto-increment): Identificador único
- **linea1** (Varchar(255)): Dirección principal (requerida)
- **linea2** (Varchar(255), NULL): Dirección secundaria (ej: apartamento, apto)
- **municipio_codigo_dane** (Varchar(10), NULL): Código oficial DANE del municipio
- **municipio_nombre** (Varchar(100)): Nombre del municipio para búsqueda/display
- **departamento** (Varchar(50), DEFAULT 'Antioquia'): Departamento fijo

- **pais** (Varchar(50), DEFAULT 'Colombia'): País fijo
- **codigo_postal** (Varchar(10), NULL): Código postal
- **referencias** (Text, NULL): Referencias adicionales (ej: "frente al parque")

REMITENTES

- **id** (PK, Auto-increment): Identificador único
- **nombre_completo** (Varchar(255)): Nombre del remitente
- **documento** (Varchar(20), NULL): Cédula o número de identificación
- **email** (Varchar(100), NULL): Correo electrónico
- **telefono** (Varchar(20), NULL): Número de teléfono
- **direccion_id** (FK, NOT NULL): Referencia a DIRECCIONES

DESTINATARIOS

- **id** (PK, Auto-increment): Identificador único
- **nombre_completo** (Varchar(255)): Nombre del destinatario
- **documento** (Varchar(20), NULL): Cédula o número de identificación
- **email** (Varchar(100), NULL): Correo electrónico
- **telefono** (Varchar(20), NULL): Número de teléfono
- **direccion_id** (FK, NOT NULL): Referencia a DIRECCIONES

ENVIOS

- **id** (PK, Auto-increment): Identificador único
- **numero_guia** (Varchar(50), UNIQUE): Guía única del envío
- **estado_envio_id** (FK, NOT NULL): Referencia a ESTADOS_ENVIO
- **remitente_id** (FK, NOT NULL): Referencia a REMITENTES
- **destinatario_id** (FK, NOT NULL): Referencia a DESTINATARIOS
- **descripcion_paquete** (Text, NULL): Descripción del contenido
- **peso_kg** (Decimal(10,2), NULL): Peso del paquete
- **fecha_creacion** (TIMESTAMP): Fecha de creación del envío
- **fecha_estimada_entrega** (Date, NULL): Fecha estimada de entrega
- **codigo_sede_registro** (Varchar(20), NULL): Código de sede (validar con tabla Seguimiento)
- **creado_por_usuario_id_externo** (UUID, NULL): ID externo del usuario que creó el envío
- **updated_at** (TIMESTAMP): Última actualización

AUDITORIA_ENVIOS

- **id** (PK, Auto-increment): Identificador único
- **envio_id** (FK, NOT NULL): Referencia a ENVIOS
- **accion** (Varchar(50)): CREATE, UPDATE, DELETE, etc.
- **usuario_id_externo** (UUID, NULL): ID externo del usuario que realizó la acción
- **valores_anteriores** (JSONB, NULL): Estado anterior en formato JSON
- **valores_nuevos** (JSONB, NULL): Nuevo estado en formato JSON

- **created_at** (TIMESTAMP): Fecha de la acción de auditoría
-

4. Restricciones de Integridad

Claves Primarias (PK)

- ESTADOS_ENVIO.id
- DIRECCIONES.id
- REMITENTES.id
- DESTINATARIOS.id
- ENVIOS.id
- AUDITORIA_ENVIOS.id

Claves Únicas (UNIQUE)

- ESTADOS_ENVIO.codigo (ej: CREADO, EN_TRANSITO)
- ENVIOS.numero_guia

Claves Foráneas (FK)

- ENVIOS.estado_envio_id → ESTADOS_ENVIO.id (CASCADE DELETE)
- ENVIOS.remitente_id → REMITENTES.id (RESTRICT DELETE)
- ENVIOS.destinatario_id → DESTINATARIOS.id (RESTRICT DELETE)
- REMITENTES.direccion_id → DIRECCIONES.id (SET NULL o RESTRICT)
- DESTINATARIOS.direccion_id → DIRECCIONES.id (SET NULL o RESTRICT)
- AUDITORIA_ENVIOS.envio_id → ENVIOS.id (CASCADE DELETE)

Restricciones de Dominio

- ENVIOS.peso_kg > 0 (si no es NULL)
 - DIRECCIONES.departamento = 'Antioquia'
 - DIRECCIONES.pais = 'Colombia'
 - REMITENTES.email válido (si no es NULL)
 - DESTINATARIOS.email válido (si no es NULL)
-

5. Notas de Diseño

Normalización

- **Forma Normal de Boyce-Codd (BCNF):** Todas las entidades están en BCNF
- **Sin redundancia:** Las direcciones se normalizan en tabla separada para evitar duplicación
- **Flexibilidad:** Se permite NULL en campos opcionales para adaptarse a diferentes escenarios

Consideraciones de Datos

1. **Direcciones:** municipio_codigo_dane se valida contra tabla externa de municipios (sin FK)
2. **Auditoria:** JSONB permite rastrear cambios complejos de atributos
3. **Estados:** Predefinidos y limitados (CREADO, EN_TRANSITO, EN_SEDE, ENTREGADO, INCIDENCIA)
4. **Usuarios:** creado_por_usuario_id_externo es UUID del sistema Auth (sin FK a BDs)

Escalabilidad

- Índices recomendados:
 - ENVIOS.numero_guia (acceso directo)
 - ENVIOS.estado_envio_id (filtrado por estado)
 - REMITENTES.documento (búsqueda de remitente)
 - DESTINATARIOS.documento (búsqueda de destinatario)
 - AUDITORIA_ENVIOS.envio_id (auditoría por envío)
 - AUDITORIA_ENVIOS.created_at (rango temporal)

6. Mapeo a Backend (Spring Boot)

Tabla	Entidad JPA	DTO	Controlador
ESTADOS_EN VIO	ShipmentState	-	-
DIRECCIONES	Address	Address Request/Response	EnvioController
REMITENTES	Sender/Party	PartyRequest/Response	EnvioController
DESTINATARIOS	Recipient/Party	PartyRequest/Response	EnvioController
ENVIOS	Shipment	CreateEnvioRequest/ShipmentResponse	EnvioController
AUDITORIA_ENVIOS	ShipmentAudit	-	-

7. Casos de Uso desde el Modelo

UC1: Registrar Envío

1. Crear REMITENTE con DIRECCIÓN
2. Crear DESTINATARIO con DIRECCIÓN
3. Crear ENVIO en estado CREADO
4. Crear registro en AUDITORIA_ENVIOS

UC2: Cambiar Estado

1. Actualizar ENVIOS.estado_envio_id
2. Registrar cambio en AUDITORIA_ENVIOS con valores_anteriores y valores_nuevos

UC3: Buscar Envío

1. Query sobre ENVIOS.numero_guia
2. JOIN con REMITENTES, DESTINATARIOS, DIRECCIONES

UC4: Historial de Auditoría

1. Query sobre AUDITORIA_ENVIOS filtrando por envio_id
2. Mostrar cambios en orden temporal (created_at)

Modelo Lógico y Especificación SQL

Sistema de Gestión de Envíos - logistica-envios

1. Especificación de Tablas (DDL)

1.1 ESTADOS_ENVIO

```
CREATE TABLE IF NOT EXISTS estados_envio (  
    id INT PRIMARY KEY AUTO_INCREMENT COMMENT 'Identificador único del estado',  
    codigo VARCHAR(30) NOT NULL UNIQUE COMMENT 'Código único: CREADO, EN_TRANSITO, EN_SEDE, ENTREGADO, INCIDENCIA',  
    nombre VARCHAR(100) NOT NULL COMMENT 'Nombre legible del estado',  
    descripcion TEXT COMMENT 'Descripción detallada del estado',  
  
    CONSTRAINT uk_codigo UNIQUE(codigo),  
    INDEX idx_codigo (codigo)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci  
COMMENT='Estados posibles de un envío en el sistema';
```

-- Datos de referencia

```
INSERT INTO estados_envio (codigo, nombre, descripcion) VALUES  
( 'CREADO', 'Creado', 'El envío ha sido creado pero aún no procesado'),
```

```
('EN_TRANSITO', 'En Tránsito', 'El envío está en camino hacia su destino'),
('EN_SEDE', 'En Sede', 'El envío se encuentra en una sede de distribución'),
('ENTREGADO', 'Entregado', 'El envío ha sido entregado al destinatario'),
('INCIDENCIA', 'Incidencia', 'Existe un problema con el envío');
```

1.2 DIRECCIONES

```
CREATE TABLE IF NOT EXISTS direcciones (
  id INT PRIMARY KEY AUTO_INCREMENT COMMENT 'Identificador único de la dirección',
  linea1 VARCHAR(255) NOT NULL COMMENT 'Primera línea de la dirección (calle, número)',
  linea2 VARCHAR(255) NULL COMMENT 'Segunda línea alternativa (apto, casa, complemento)',
  municipio_codigo_dane VARCHAR(10) NULL COMMENT 'Código DANE del municipio para validación',
  municipio_nombre VARCHAR(100) NOT NULL COMMENT 'Nombre del municipio para búsqueda/display',
  departamento VARCHAR(50) NOT NULL DEFAULT 'Antioquia' COMMENT 'Departamento (fijo para operación actual)',
  pais VARCHAR(50) NOT NULL DEFAULT 'Colombia' COMMENT 'País (fijo)',
  codigo_postal VARCHAR(10) NULL COMMENT 'Código postal',
  referencias TEXT NULL COMMENT 'Referencias adicionales (ej: frente al parque, cerca a...)',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP COMMENT 'Fecha de creación del registro',
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT 'Última actualización',

  CONSTRAINT ck_departamento CHECK (departamento = 'Antioquia'),
  CONSTRAINT ck_pais CHECK (pais = 'Colombia'),
  INDEX idx_municipio (municipio_codigo_dane),
  INDEX idx_linea1_municipio (linea1, municipio_nombre),
  INDEX idx_codigo_postal (codigo_postal)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT='Direcciones de remitentes y destinatarios, normalizadas para Antioquia';
```

1.3 REMITENTES

```
CREATE TABLE IF NOT EXISTS remitentes (
  id INT PRIMARY KEY AUTO_INCREMENT COMMENT 'Identificador único del remitente',
  nombre_completo VARCHAR(255) NOT NULL COMMENT 'Nombre completo del remitente',
  documento VARCHAR(20) NULL COMMENT 'Número de cédula o documento de
```



```

identidad',
    email VARCHAR(100) NULL COMMENT 'Correo electrónico del remitente',
    telefono VARCHAR(20) NULL COMMENT 'Número de teléfono (con extensión
si aplica)',
    direccion_id INT NOT NULL COMMENT 'FK: ID de la dirección',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP COMMENT 'Fecha de
creación del registro',
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP COMMENT 'Última actualización',

    CONSTRAINT fk_remitentes_direccion FOREIGN KEY (direccion_id)
        REFERENCES direcciones(id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    CONSTRAINT ck_email_remitente CHECK (email IS NULL OR email LIKE
'%@%.%'),
    INDEX idx_documento (documento),
    INDEX idx_email (email),
    INDEX idx_direccion_id (direccion_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT='Datos del remitente del envío (quien envía)';

```

1.4 DESTINATARIOS

```

CREATE TABLE IF NOT EXISTS destinatarios (
    id INT PRIMARY KEY AUTO_INCREMENT COMMENT 'Identificador único del
destinatario',
    nombre_completo VARCHAR(255) NOT NULL COMMENT 'Nombre completo del
destinatario',
    documento VARCHAR(20) NULL COMMENT 'Número de cédula o documento de
identidad',
    email VARCHAR(100) NULL COMMENT 'Correo electrónico del
destinatario',
    telefono VARCHAR(20) NULL COMMENT 'Número de teléfono (con extensión
si aplica)',
    direccion_id INT NOT NULL COMMENT 'FK: ID de la dirección',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP COMMENT 'Fecha de
creación del registro',
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP COMMENT 'Última actualización',

    CONSTRAINT fk_destinatarios_direccion FOREIGN KEY (direccion_id)
        REFERENCES direcciones(id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    CONSTRAINT ck_email_destinatario CHECK (email IS NULL OR email LIKE
'%@%.%'),
    INDEX idx_documento (documento),
    INDEX idx_email (email),

```

```

    INDEX idx_direccion_id (direccion_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT='Datos del destinatario del envío (quien recibe)';

```

1.5 ENVÍOS

```

CREATE TABLE IF NOT EXISTS envios (
    id INT PRIMARY KEY AUTO_INCREMENT COMMENT 'Identificador único del
envío',
    numero_guia VARCHAR(50) NOT NULL UNIQUE COMMENT 'Número de guía único
(generado automáticamente)',
    estado_envio_id INT NOT NULL COMMENT 'FK: Estado actual del envío',
    remitente_id INT NOT NULL COMMENT 'FK: Remitente del envío',
    destinatario_id INT NOT NULL COMMENT 'FK: Destinatario del envío',
    descripcion_paquete TEXT NULL COMMENT 'Descripción del contenido del
paquete',
    peso_kg DECIMAL(10, 2) NULL COMMENT 'Peso del paquete en kilogramos',
    fecha_creacion TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT
'Fecha de creación del envío',
    fecha_estimada_entrega DATE NULL COMMENT 'Fecha estimada de entrega
al destinatario',
    codigo_sede_registro VARCHAR(20) NULL COMMENT 'Código de la sede
donde se registró el envío',
    creado_por_usuario_id_externo CHAR(36) NULL COMMENT 'UUID del usuario
del sistema Auth que creó el envío',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP COMMENT 'Timestamp de
creación en BD',
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP COMMENT 'Última actualización',

```

```

    CONSTRAINT fk_envios_estado FOREIGN KEY (estado_envio_id)
        REFERENCES estados_envio(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_envios_remitente FOREIGN KEY (remitente_id)
        REFERENCES remitentes(id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    CONSTRAINT fk_envios_destinatario FOREIGN KEY (destinatario_id)
        REFERENCES destinatarios(id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    CONSTRAINT ck_peso_positivo CHECK (peso_kg IS NULL OR peso_kg > 0),
    CONSTRAINT ck_fecha_entrega CHECK (fecha_estimada_entrega IS NULL OR
fecha_estimada_entrega >= CAST(fecha_creacion AS DATE)),
    CONSTRAINT uk_numero_guia UNIQUE(numero_guia),
    INDEX idx_numero_guia (numero_guia),
    INDEX idx_estado_envio_id (estado_envio_id),
    INDEX idx_remitente_id (remitente_id),

```

```

INDEX idx_destinatario_id (destinatario_id),
INDEX idx_fecha_creacion (fecha_creacion),
INDEX idx_estado_fecha (estado_envio_id, fecha_creacion),
INDEX idx_usuario_creador (creado_por_usuario_id_externo),
INDEX idx_sede_registro (codigo_sede_registro)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT='Tabla principal de envíos con información consolidada de
remitente, destinatario y estado';

```

1.6 AUDITORIA_ENVIOS

```

CREATE TABLE IF NOT EXISTS auditoria_envios (
  id INT PRIMARY KEY AUTO_INCREMENT COMMENT 'Identificador único del
registro de auditoría',
  envio_id INT NOT NULL COMMENT 'FK: ID del envío auditado',
  accion VARCHAR(50) NOT NULL COMMENT 'Tipo de acción: CREATE, UPDATE,
DELETE, ESTADO_CAMBIO, etc.',
  usuario_id_externo CHAR(36) NULL COMMENT 'UUID del usuario que
realizó la acción (desde Auth)',
  valores_anteriores JSON NULL COMMENT 'Estado anterior del registro
(JSONB compatible)',
  valores_nuevos JSON NULL COMMENT 'Nuevo estado del registro (JSONB
compatible)',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP COMMENT 'Timestamp de
la acción de auditoría',

  CONSTRAINT fk_auditoria_envio FOREIGN KEY (envio_id)
    REFERENCES envios(id)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
  INDEX idx_envio_id (envio_id),
  INDEX idx_accion (accion),
  INDEX idx_usuario (usuario_id_externo),
  INDEX idx_created_at (created_at),
  INDEX idx_envio_fecha (envio_id, created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT='Registro de cambios sensibles en los envíos para auditoría y
trazabilidad administrativa';

```

2. Especificación de Índices

Índices Críticos para Rendimiento

Tabla	Nombre	Columnas	Tipo	Justificación
envios	idx_numero_ guia	numero_guia	UNIQUE	Consultas directas por guía
envios	idx_estado_e	estado_envio_id	BTREE	Filtrado por estado

Tabla	Nombre	Columnas	Tipo	Justificación
	envio_id			
envios	idx_fecha_creacion	fecha_creacion	BTREE	Rangos de fechas
envios	idx_estado_fecha	(estado_envio_id, fecha_creacion)	BTREE	Combinadas: estado reciente
remitentes	idx_documento	documento	BTREE	Búsqueda de remitente
destinatarios	idx_documento	documento	BTREE	Búsqueda de destinatario
direcciones	idx_municipio	municipio_codigo_dane	BTREE	Validación de municipios
auditoria_envios	idx_envio_fecha	(envio_id, created_at)	BTREE	Historial ordenado

3. Vistas SQL Útiles

Vista: Envíos Completos

```

CREATE VIEW IF NOT EXISTS vw_envios_completos AS
SELECT
    e.id,
    e.numero_guia,
    e.fecha_creacion,
    ee.codigo AS estado_codigo,
    ee.nombre AS estado_nombre,
    r.nombre_completo AS remitente_nombre,
    r.documento AS remitente_documento,
    r.telefono AS remitente_telefono,
    CONCAT_WS(' ', dr.linea1, dr.linea2, dr.municipio_nombre,
dr.departamento) AS remitente_direccion,
    d.nombre_completo AS destinatario_nombre,
    d.documento AS destinatario_documento,
    d.telefono AS destinatario_telefono,
    CONCAT_WS(' ', dd.linea1, dd.linea2, dd.municipio_nombre,
dd.departamento) AS destinatario_direccion,
    e.descripcion_paquete,
    e.peso_kg,
    e.fecha_estimada_entrega,
    e.codigo_sede_registro,
    e.updated_at
FROM envios e
JOIN estados_envio ee ON e.estado_envio_id = ee.id
JOIN remitentes r ON e.remitente_id = r.id
JOIN direcciones dr ON r.direccion_id = dr.id
JOIN destinatarios d ON e.destinatario_id = d.id

```

```
JOIN direcciones dd ON d.direccion_id = dd.id
ORDER BY e.fecha_creacion DESC;
```

Vista: Estado de Envíos por Municipio

```
CREATE VIEW IF NOT EXISTS vw_envios_por_municipio AS
SELECT
    dd.municipio_nombre AS municipio_destino,
    ee.nombre AS estado,
    COUNT(e.id) AS cantidad_envios,
    AVG(e.peso_kg) AS peso_promedio_kg,
    MAX(e.fecha_creacion) AS ultimo_registro
FROM envios e
JOIN estados_envio ee ON e.estado_envio_id = ee.id
JOIN destinatarios d ON e.destinatario_id = d.id
JOIN direcciones dd ON d.direccion_id = dd.id
GROUP BY dd.municipio_nombre, ee.nombre
ORDER BY dd.municipio_nombre, cantidad_envios DESC;
```

Vista: Historial de Cambios de Estado

```
CREATE VIEW IF NOT EXISTS vw_historial_cambios_estado AS
SELECT
    ae.envio_id,
    e.numero_guia,
    ae.created_at,
    JSON_UNQUOTE(JSON_EXTRACT(ae.valores_anteriores,
    '$.estado_envio_id')) AS estado_anterior_id,
    JSON_UNQUOTE(JSON_EXTRACT(ae.valores_nuevos, '$.estado_envio_id')) AS
estado_nuevo_id,
    ae.usuario_id_externo,
    TIMESTAMPDIFF(HOUR,
        DATE_SUB(ae.created_at, INTERVAL (
            SELECT IFNULL(ROW_NUMBER() OVER (PARTITION BY envio_id ORDER
BY created_at), 0)
        ) HOUR),
        ae.created_at
    ) AS horas_en_estado_anterior
FROM auditoria_envios ae
JOIN envios e ON ae.envio_id = e.id
WHERE ae.accion = 'ESTADO_CAMBIO'
ORDER BY ae.envio_id, ae.created_at;
```

4. Procedimientos Almacenados Recomendados

SP: Registrar Cambio de Estado

```
DELIMITER //
```

```
CREATE PROCEDURE IF NOT EXISTS sp_cambiar_estado_envio(
    IN p_envio_id INT,
```

```

    IN p_nuevo_estado_id INT,
    IN p_usuario_id_externo CHAR(36),
    OUT p_exito BOOLEAN,
    OUT p_mensaje VARCHAR(500)
)
READS SQL DATA MODIFIES SQL DATA
COMMENT 'Cambia el estado de un envío y registra en auditoría'
BEGIN
    DECLARE v_estado_anterior_id INT;
    DECLARE v_numero_guia VARCHAR(50);

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        SET p_exito = FALSE;
        SET p_mensaje = 'Error al cambiar estado del envío';
        ROLLBACK;
    END;

    START TRANSACTION;

    -- Obtener estado anterior
    SELECT estado_envio_id INTO v_estado_anterior_id
    FROM envios
    WHERE id = p_envio_id
    FOR UPDATE;

    IF v_estado_anterior_id IS NULL THEN
        SET p_exito = FALSE;
        SET p_mensaje = 'Envío no encontrado';
        ROLLBACK;
    ELSE
        -- Actualizar estado
        UPDATE envios
        SET estado_envio_id = p_nuevo_estado_id,
            updated_at = CURRENT_TIMESTAMP
        WHERE id = p_envio_id;

        -- Registrar en auditoría
        INSERT INTO auditoria_envios (envio_id, accion,
usuario_id_externo, valores_anteriores, valores_nuevos)
        VALUES (
            p_envio_id,
            'ESTADO_CAMBIO',
            p_usuario_id_externo,
            JSON_OBJECT('estado_envio_id', v_estado_anterior_id),
            JSON_OBJECT('estado_envio_id', p_nuevo_estado_id)
        );

        SET p_exito = TRUE;
    
```

```

        SET p_mensaje = 'Estado cambió exitosamente';
        COMMIT;
    END IF;
END //

```

```

DELIMITER ;

```

SP: Registrar Nuevo Envío

```

DELIMITER //

```

```

CREATE PROCEDURE IF NOT EXISTS sp_crear_envio(
    IN p_numero_guia VARCHAR(50),
    IN p_remitente_id INT,
    IN p_destinatario_id INT,
    IN p_usuario_id_externo CHAR(36),
    IN p_descripcion_paquete TEXT,
    IN p_peso_kg DECIMAL(10,2),
    IN p_codigo_sede_registro VARCHAR(20),
    OUT p_envio_id INT,
    OUT p_exito BOOLEAN,
    OUT p_mensaje VARCHAR(500)
)
READS SQL DATA MODIFIES SQL DATA
COMMENT 'Crea un nuevo envío con auditoría'
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        SET p_exito = FALSE;
        SET p_mensaje = 'Error al crear el envío';
        SET p_envio_id = NULL;
        ROLLBACK;
    END;

    START TRANSACTION;

    INSERT INTO envios (
        numero_guia,
        estado_envio_id,
        remitente_id,
        destinatario_id,
        descripcion_paquete,
        peso_kg,
        codigo_sede_registro,
        creado_por_usuario_id_externo
    ) VALUES (
        p_numero_guia,
        (SELECT id FROM estados_envio WHERE codigo = 'CREADO'),
        p_remitente_id,
        p_destinatario_id,
        p_descripcion_paquete,

```

```

        p_peso_kg,
        p_codigo_sede_registro,
        p_usuario_id_externo
    );

    SET p_envio_id = LAST_INSERT_ID();

    -- Registrar en auditoría
    INSERT INTO auditoria_envios (envio_id, accion, usuario_id_externo,
valores_nuevos)
    VALUES (
        p_envio_id,
        'CREATE',
        p_usuario_id_externo,
        JSON_OBJECT(
            'numero_guia', p_numero_guia,
            'descripcion_paquete', p_descripcion_paquete,
            'peso_kg', p_peso_kg
        )
    );

    SET p_exito = TRUE;
    SET p_mensaje = CONCAT('Envío creado con ID: ', p_envio_id);
    COMMIT;
END //

DELIMITER ;

```

5. Características de Seguridad

Validaciones a Nivel de BD

1. **Restricción de Dominio (CHECK):**
 - peso_kg > 0 (si existe)
 - fecha_estimada_entrega >= fecha_creacion
 - departamento = 'Antioquia'
 - pais = 'Colombia'
 - Email válido (formato básico)
2. **Integridad Referencial:**
 - FK con ON DELETE RESTRICT para mantener datos relacionados
 - FK ON DELETE CASCADE para auditoría (se elimina con envío)
 - ON UPDATE CASCADE para cambios de PK
3. **Unicidad:**
 - numero_guia UNIQUE (no puede haber dos envíos con el mismo número)
 - codigo en estados_envio UNIQUE

Auditoría

- Tabla auditoria_envios registra:
 - QUIÉN cambió (usuario_id_externo)
 - QUÉ cambió (valores_anteriores, valores_nuevos)
 - CUÁNDO cambió (created_at)
 - QUÉ tipo de acción (CREATE, UPDATE, DELETE, ESTADO_CAMBIO)
-

6. Especificación de Collations y Character Sets

-- Base de datos

```
CREATE DATABASE IF NOT EXISTS logistica_envios
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
```

-- Utilizar en todas las tablas:

```
CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci
```

Justificación: UTF8MB4 soporta caracteres especiales españoles (acentos, ñ) y emojis si es necesario en el futuro.

7. Script de Creación Completo

-- Ejecutar en orden:

-- 1. Crear tabla de estados

```
SOURCE /path/to/01_estados_envio.sql;
```

-- 2. Crear tabla de direcciones

```
SOURCE /path/to/02_direcciones.sql;
```

-- 3. Crear tabla de remitentes

```
SOURCE /path/to/03_remitentes.sql;
```

-- 4. Crear tabla de destinatarios

```
SOURCE /path/to/04_destinatarios.sql;
```

-- 5. Crear tabla de envíos

```
SOURCE /path/to/05_envios.sql;
```

-- 6. Crear tabla de auditoría

```
SOURCE /path/to/06_auditoria_envios.sql;
```

-- 7. Crear vistas

```
SOURCE /path/to/07_vistas.sql;
```

-- 8. Crear procedimientos almacenados

```
SOURCE /path/to/08_procedimientos.sql;
```

-- 9. Crear índices adicionales
SOURCE /path/to/09_indices.sql;

8. Tabla de Normalización

Tabla	1NF	2NF	3NF	BCNF	Notas
estados_envio	✓	✓	✓	✓	Campo clave simple (id)
direcciones	✓	✓	✓	✓	Todos atributos dependen de id
remitentes	✓	✓	✓	✓	FK solo a direcciones
destinatarios	✓	✓	✓	✓	FK solo a direcciones
envios	✓	✓	✓	✓	Todos atributos dependen de id primario
auditoria_envios	✓	✓	✓	✓	Registro de eventos, atributos JSON

9. Consideraciones de Base de Datos

Engine

- **InnoDB:** Soporte de transacciones, integridad referencial, mejor manejo de concurrencia

Backup

- Realizar backup diario, especialmente de auditoria_envios para trazabilidad
- Retener auditoría por mínimo 1 año (legal)

Particionamiento (Futuro)

Si el volumen crece, particionar:

-- Ejemplo: Particionar envios por año de creación
ALTER TABLE envios **PARTITION BY RANGE** (YEAR(fecha_creacion)) (
 PARTITION p2024 **VALUES LESS THAN** (2025),
 PARTITION p2025 **VALUES LESS THAN** (2026),
 PARTITION p_future **VALUES LESS THAN MAXVALUE**
);

10. Ejemplos de Consultas Comunes

Obtener envío con todos sus detalles

```
SELECT * FROM vw_envios_completos
WHERE numero_guia = 'ECV-2026-001234';
```

Contar envíos por estado en última semana

```
SELECT
    ee.nombre AS estado,
    COUNT(e.id) AS cantidad
FROM envios e
JOIN estados_envio ee ON e.estado_envio_id = ee.id
WHERE e.fecha_creacion >= DATE_SUB(NOW(), INTERVAL 7 DAY)
GROUP BY ee.nombre
ORDER BY cantidad DESC;
```

Historial completo de cambios en un envío

```
SELECT
    ae.accion,
    ae.created_at,
    ae.usuario_id_externo,
    ae.valores_anteriores,
    ae.valores_nuevos
FROM auditoria_envios ae
WHERE ae.envio_id = 1
ORDER BY ae.created_at ASC;
```

Buscar direcciones duplicadas

```
SELECT
    linea1,
    municipio_nombre,
    COUNT(*) AS cantidad
FROM direcciones
GROUP BY linea1, municipio_nombre
HAVING cantidad > 1
ORDER BY cantidad DESC;
```

Modelo Físico de Base de Datos

Sistema: Envíos (logística-envios)

1. Alcance físico

Este modelo físico está aterrizado para PostgreSQL y define cómo se implementa el esquema en disco y en ejecución:

- Tipos de datos concretos (UUID, TIMESTAMPTZ, JSONB, NUMERIC)
- Claves primarias, foráneas y restricciones CHECK
- Índices de rendimiento para consultas operativas
- Convenciones de nombres para producción
- Script SQL ejecutable por migraciones

2. Motor y configuración recomendada

- Motor: PostgreSQL 14+
- Encoding: UTF-8
- Timezone de base de datos: America/Bogota
- Extensiones recomendadas:

```
CREATE EXTENSION IF NOT EXISTS pgcrypto; -- gen_random_uuid()
```

3. Esquema físico (DDL)

```
-- =====
-- ESQUEMA FISICO - BD ENVIOS
-- =====
```

```
CREATE TABLE IF NOT EXISTS estados_envio (
    id UUID PRIMARY KEY,
    codigo VARCHAR(50) NOT NULL UNIQUE,
    nombre VARCHAR(120) NOT NULL,
    descripcion VARCHAR(500)
);
```

```
CREATE TABLE IF NOT EXISTS direcciones (
    id UUID PRIMARY KEY,
    linea1 VARCHAR(255) NOT NULL,
    linea2 VARCHAR(255),
    municipio_codigo_dane VARCHAR(5),
    municipio_nombre VARCHAR(120),
    departamento VARCHAR(80) NOT NULL DEFAULT 'Antioquia',
    pais VARCHAR(80) NOT NULL DEFAULT 'Colombia',
    codigo_postal VARCHAR(20),
    referencias VARCHAR(500),
    CONSTRAINT ck_direcciones_departamento CHECK (departamento =
'Antioquia'),
    CONSTRAINT ck_direcciones_pais CHECK (pais = 'Colombia')
);
```

```
CREATE TABLE IF NOT EXISTS remitentes (
    id UUID PRIMARY KEY,
    nombre_completo VARCHAR(200) NOT NULL,
    documento VARCHAR(50),
    email VARCHAR(255),
    telefono VARCHAR(50),
    direccion_id UUID NOT NULL,
    CONSTRAINT fk_remitentes_direccion
```

```

        FOREIGN KEY (direccion_id) REFERENCES direcciones(id),
        CONSTRAINT ck_remitentes_email
        CHECK (email IS NULL OR email ~* '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-
]+\\.[A-Za-z]{2,}$')
);

```

```

CREATE TABLE IF NOT EXISTS destinatarios (
    id UUID PRIMARY KEY,
    nombre_completo VARCHAR(200) NOT NULL,
    documento VARCHAR(50),
    email VARCHAR(255),
    telefono VARCHAR(50),
    direccion_id UUID NOT NULL,
    CONSTRAINT fk_destinatarios_direccion
    FOREIGN KEY (direccion_id) REFERENCES direcciones(id),
    CONSTRAINT ck_destinatarios_email
    CHECK (email IS NULL OR email ~* '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-
]+\\.[A-Za-z]{2,}$')
);

```

```

CREATE TABLE IF NOT EXISTS envios (
    id UUID PRIMARY KEY,
    numero_guia VARCHAR(32) NOT NULL UNIQUE,
    estado_envio_id UUID NOT NULL,
    remitente_id UUID NOT NULL,
    destinatario_id UUID NOT NULL,
    descripcion_paquete VARCHAR(500),
    peso_kg NUMERIC(10,2),
    fecha_creacion TIMESTAMPTZ NOT NULL,
    fecha_estimada_entrega DATE,
    codigo_sede_registro VARCHAR(50),
    creado_por_usuario_id_externo UUID,
    updated_at TIMESTAMPTZ NOT NULL,

    CONSTRAINT fk_envios_estado
    FOREIGN KEY (estado_envio_id) REFERENCES estados_envio(id),
    CONSTRAINT fk_envios_remitente
    FOREIGN KEY (remitente_id) REFERENCES remitentes(id),
    CONSTRAINT fk_envios_destinatario
    FOREIGN KEY (destinatario_id) REFERENCES destinatarios(id),

    CONSTRAINT ck_envios_peso_positivo
    CHECK (peso_kg IS NULL OR peso_kg > 0),
    CONSTRAINT ck_envios_fecha_entrega
    CHECK (fecha_estimada_entrega IS NULL OR fecha_estimada_entrega
>= fecha_creacion::date)
);

```

```

CREATE TABLE IF NOT EXISTS auditoria_envios (
    id UUID PRIMARY KEY,

```

```

    envio_id UUID NOT NULL,
    accion VARCHAR(80) NOT NULL,
    usuario_id_externo UUID,
    valores_anteriores JSONB,
    valores_nuevos JSONB,
    created_at TIMESTAMPTZ NOT NULL,

    CONSTRAINT fk_auditoria_envio
        FOREIGN KEY (envio_id) REFERENCES envios(id)
);

```

4. Índices físicos

-- Índices existentes y recomendados

```

CREATE INDEX IF NOT EXISTS idx_envios_estado ON envios (estado_envio_id);
CREATE INDEX IF NOT EXISTS idx_envios_fecha_creacion ON envios
(fecha_creacion DESC);
CREATE INDEX IF NOT EXISTS idx_envios_numero_guia ON envios
(numero_guia);
CREATE INDEX IF NOT EXISTS idx_envios_estado_fecha ON envios
(estado_envio_id, fecha_creacion DESC);

```

```

CREATE INDEX IF NOT EXISTS idx_remitentes_documento ON remitentes
(documento);
CREATE INDEX IF NOT EXISTS idx_destinatarios_documento ON destinatarios
(documento);
CREATE INDEX IF NOT EXISTS idx_direcciones_municipio_dane ON direcciones
(municipio_codigo_dane);

```

```

CREATE INDEX IF NOT EXISTS idx_auditoria_envio ON auditoria_envios
(envio_id);
CREATE INDEX IF NOT EXISTS idx_auditoria_envio_fecha ON auditoria_envios
(envio_id, created_at DESC);
CREATE INDEX IF NOT EXISTS idx_auditoria_accion_fecha ON auditoria_envios
(accion, created_at DESC);

```

5. Semillas físicas de catálogo

```

INSERT INTO estados_envio (id, codigo, nombre, descripcion) VALUES
('a0000001-0000-4000-8000-000000000001', 'CREADO', 'Creado',
'Radicado en el sistema'),
('a0000001-0000-4000-8000-000000000002', 'EN_TRANSITO', 'En
tránsito', 'En movimiento entre nodos'),
('a0000001-0000-4000-8000-000000000003', 'EN_SEDE', 'En sede', 'En
oficina o hub'),
('a0000001-0000-4000-8000-000000000004', 'ENTREGADO', 'Entregado',
'Entrega confirmada'),
('a0000001-0000-4000-8000-000000000005', 'INCIDENCIA', 'Incidencia',
'Requiere atención')
ON CONFLICT (id) DO NOTHING;

```

6. Políticas operativas del modelo físico

- auditoria_envios es append-only para trazabilidad.
- Los UUID se generan en aplicación o con gen_random_uuid().
- envios.numero_guia es clave de negocio única para tracking.
- municipio_codigo_dane se valida por API externa (sin FK inter-bd).

7. Consultas físicas típicas (optimizadas)

-- Tracking por guía (usa índice unique)

```
SELECT *  
FROM envios  
WHERE numero_guia = 'ECV-2026-0001234';
```

-- Bandeja operativa por estado y recencia (usa índice compuesto)

```
SELECT e.id, e.numero_guia, e.fecha_creacion  
FROM envios e  
WHERE e.estado_envio_id = 'a0000001-0000-4000-8000-000000000002'  
ORDER BY e.fecha_creacion DESC  
LIMIT 100;
```

-- Historial de auditoría por envío

```
SELECT a.accion, a.valores_anteriores, a.valores_nuevos, a.created_at  
FROM auditoria_envios a  
WHERE a.envio_id = '11111111-1111-4111-8111-111111111111'  
ORDER BY a.created_at DESC;
```

8. Diferencia con el modelo lógico

- Modelo lógico: qué entidades y relaciones hay.
- Modelo físico: cómo se implementa en PostgreSQL (tipos, índices, constraints, ejecución real).

Este documento representa la capa de implementación real en base de datos para el dominio de envíos.